

CoMERA: **C**omputing- and **M**emory-**E**fficient Training via **R**ank-**A**daptive Tensor Optimization

Zi Yang¹, Ziyue Liu², Samridhi Choudhary³, Xinfeng Xie⁴, Cao Gao⁴,
Siegfried Kunzmann³, Zheng Zhang²

1. University at Albany, SUNY; 2. UC Santa Barbara; 3. Amazon Alexa AI; 4. Meta.

UC SANTA BARBARA



1. Introduction

The cost of training large AI models is exploding!
Amortized hardware and energy cost to train frontier AI models over time

EPOCH AI

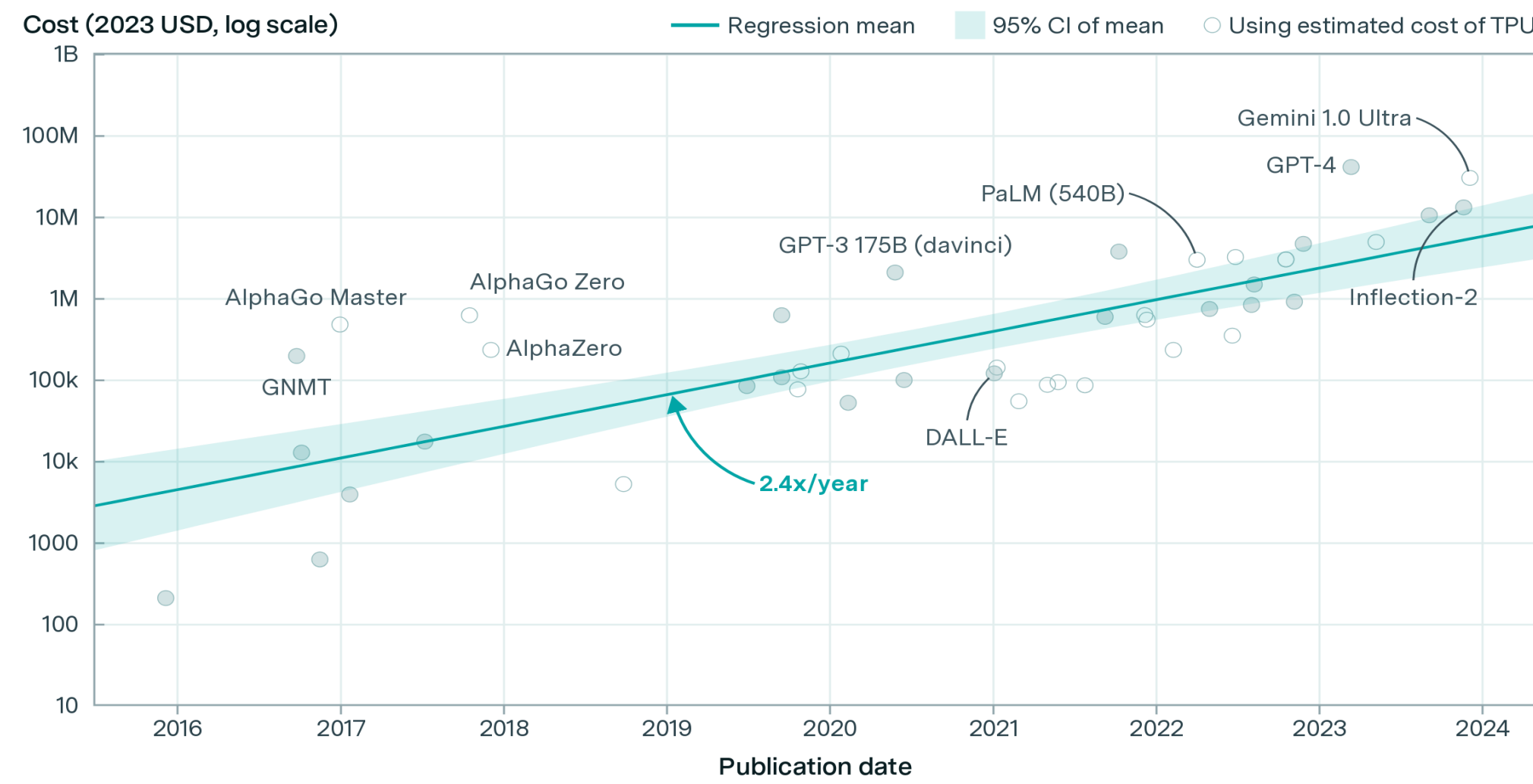


Figure from: <https://epochai.org/blog/how-much-does-it-cost-to-train-frontier-ai-models>

Contributions

We propose CoMERA for **Efficient Training** via **Tensor Compression**.

- **Multi-Objective Optimization for Rank-Adaptive Tensor-Training:**
 - Balance compression ratio and model performance
 - Customize model for a specific resource requirement
 - Partially capable of automatic architecture search
- **Performance Optimization of Tensor-Compressed Training:**
 - Optimize tensor-network contractions of tensorized linear layers and lookup of tensorized embedding tables
 - Eliminate the GPU backend overhead via CUDA Graph

2. CoMERA Training Framework

Multi-Objective Training Model

- **Modified Tensor-Train representation of weights:** A weight matrix W_i is reshaped into a tensor $\mathcal{W}$ of size $n_1 \times \dots \times n_{2d}$ and represented as
$$\mathcal{W} = \mathcal{G}_1 \times_{3,1} D_1 \times_{2,1} \mathcal{G}_2 \times_{3,1} \dots \times_{3,1} D_{2d-1} \times \mathcal{G}_{2d},$$
where $\mathcal{G}_i \in \mathbb{R}^{r_{i-1} \times n_i \times r_i}$'s are tensor cores and $D_i \in \mathbb{R}^{r_i \times r_i}$'s are diagonal matrices controlling TT ranks.
- **Multi-objective training:** The multi-objective training minimizes both the loss and compressed network size: $\min_{\mathcal{G}, D} (L(\mathcal{G}, D), S(D))$, where $L(\mathcal{G}, D)$ is the model loss and $S(D)$ is the model size controlled by D . We look for a Pareto point $(\mathcal{G}^*, D^*)$ of the problem, i.e. there does not exist $(\mathcal{G}, D)$ such that $L(\mathcal{G}, D) \leq L(\mathcal{G}^*, D^*)$ and $S(\mathcal{G}, D) < S(\mathcal{G}^*, D^*)$ and at least one of them is strict.

Two-Stage Training Method:

- **Early-stage:** We start the training with the following linear scalarization formulation to gradually prune tensor ranks.

$$\min_{\mathcal{G}, D} L(\mathcal{G}, D) + \gamma \hat{S}(D) + \beta \|\mathcal{G}\|^2,$$

where $\hat{S}(D)$ is the l_1 -relaxation of $S(D)$ and $\beta \|\mathcal{G}\|^2$ term controls magnitude of $\mathcal{G}$.

- **Late-stage (optional):** Given target loss L_0 and target size S_0 , we solve the following achievement scalarization for a Pareto point close to (L_0, S_0)

$$\min_{\mathcal{G}, D} \max\{\omega_1(L(\mathcal{G}, D) - L_0), \omega_2(S(D) - S_0)\} + \rho(L(\mathcal{G}, D) + \hat{S}(D)).$$

3. Performance Optimization

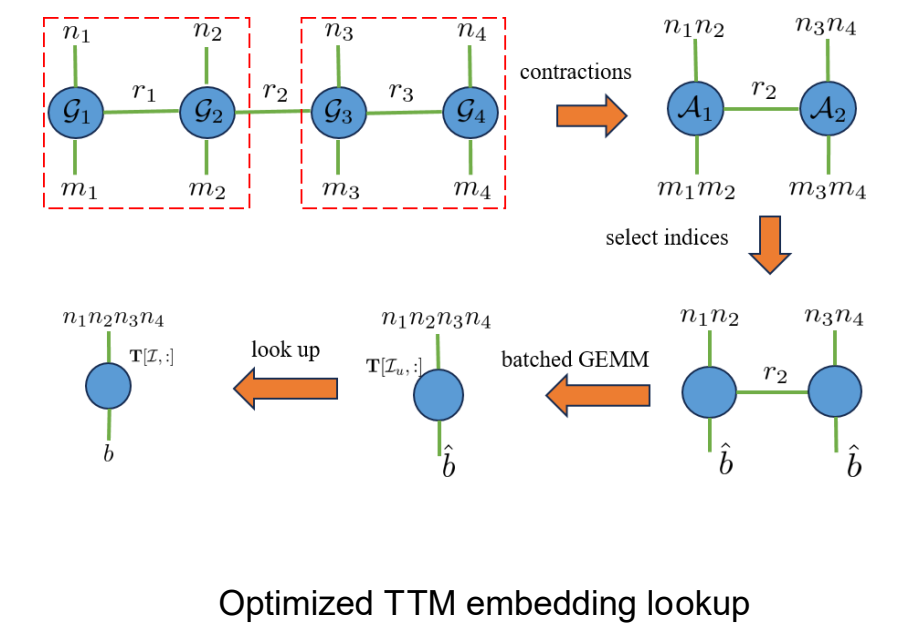
Performance Optimization of TTM Embedding Tables

- The embedding table is represented in the Tensor-Train-Matrix format,

$$\mathcal{T} = \mathcal{G}_1 \times_{4,1} \mathcal{G}_2 \times_{4,1} \mathcal{G}_3 \times_{4,1} \mathcal{G}_4$$

where $\mathcal{G}_i \in \mathbb{R}^{r_{i-1} \times m_i \times n_i \times r_i}$.

- We propose a fast and memory-efficient optimized TTM lookup by considering uniqueness on token level and tensor index level.



Optimized TTM embedding lookup

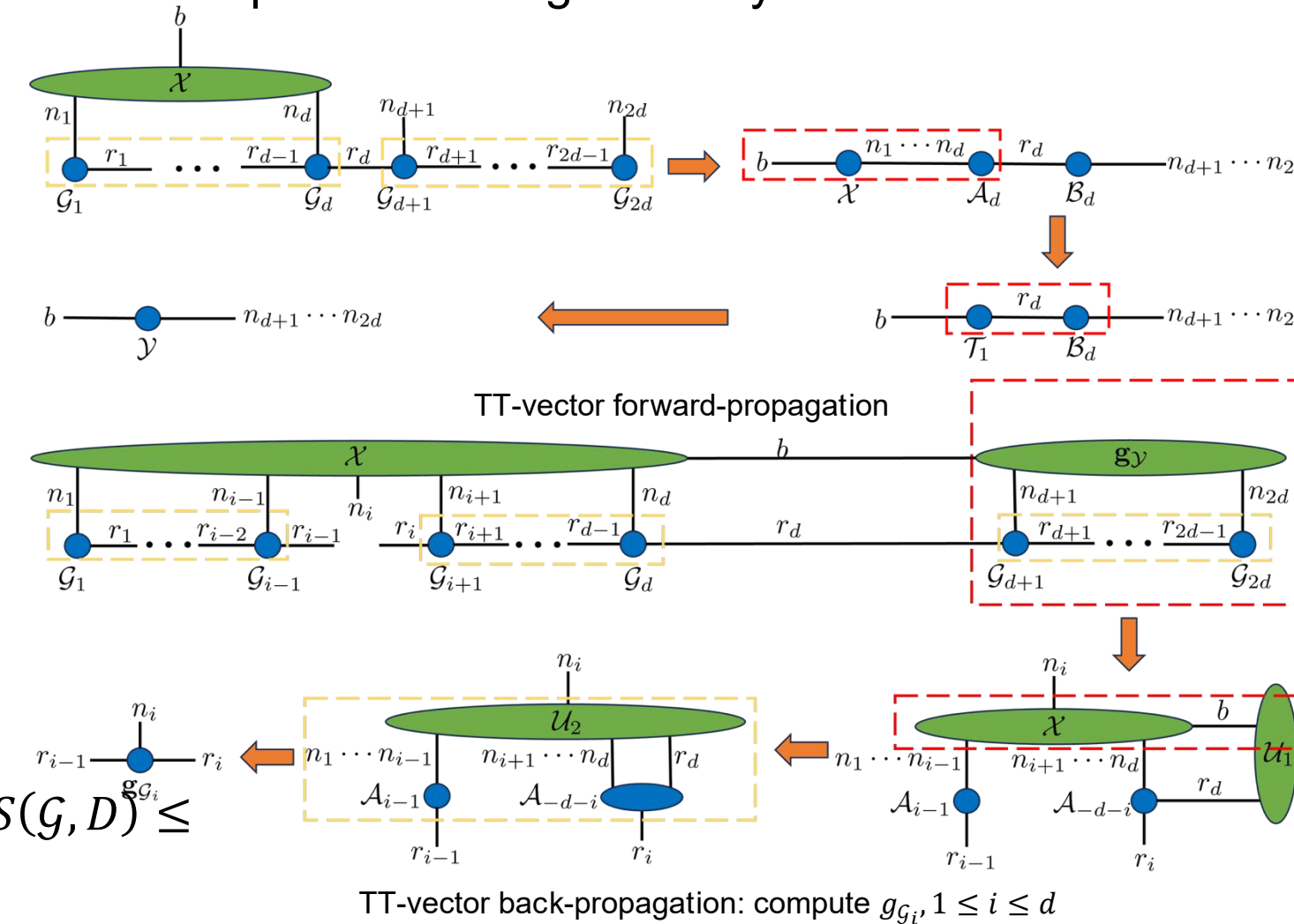
Optimized TT-vector multiplication for training

- For linear layer $Y = XW$, the weight W is expressed in the Tensor-Train format

$$\mathcal{W} = \mathcal{G}_1 \times_{3,1} \dots \times_{3,1} \mathcal{G}_{2d},$$

where $\mathcal{G}_i \in \mathbb{R}^{r_{i-1} \times n_i \times r_i}$.

- We design the forward- and back-propagation contraction paths by minimizing total number of flops and reusing as many intermediate results as possible.

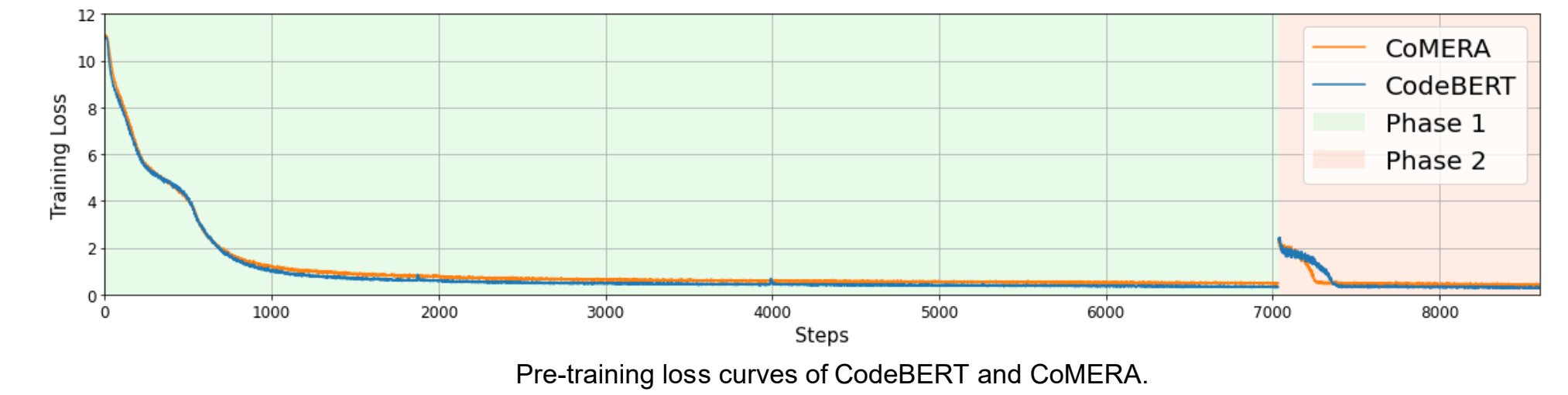


TT-vector back-propagation: compute $g_{G_i}, 1 \leq i \leq d$

4. Experiments

CodeBERT_{LARGE} of 357M parameters

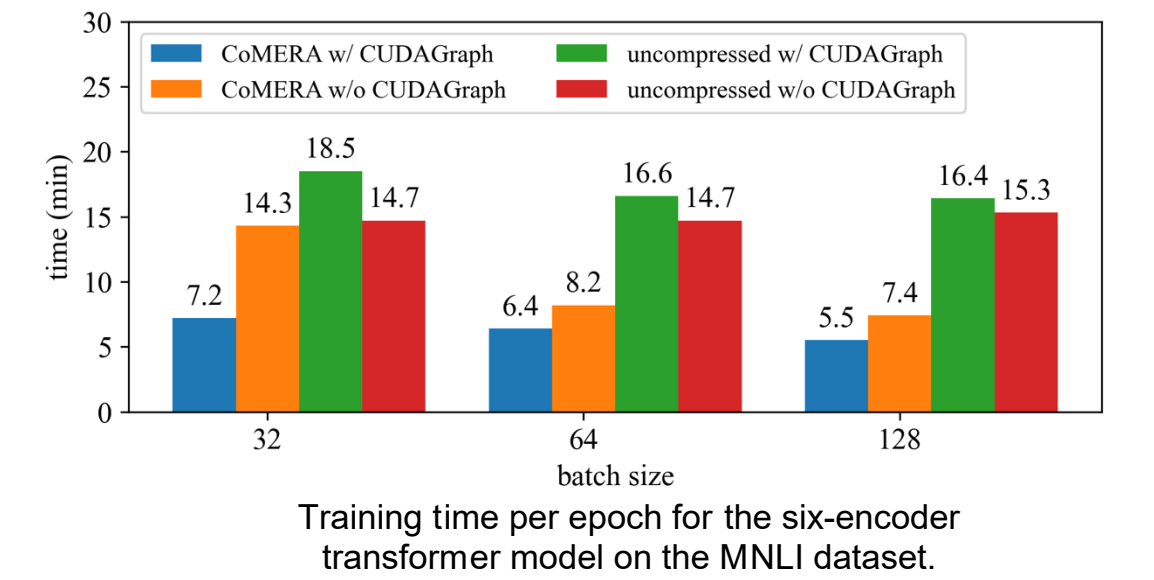
- We train CodeBERT_{LARGE} from scratch on CodeSearchNet dataset, a collection of 2M pairs and 6M pure code sequences and reach
 - A similar training loss curve
 - **4.23×** model compression ratio
 - **1.9-2.3×** training speedup in different training phases.



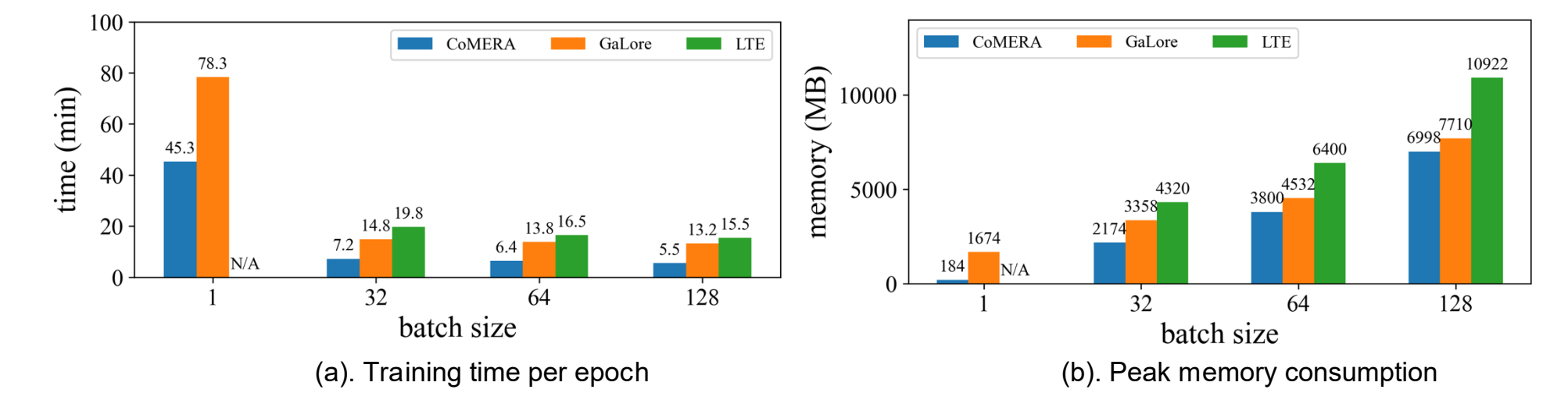
Pre-training loss curves of CodeBERT and CoMERA.

Six-encoder Transformer

- We test CoMERA to train a six-encoder Transformer on MNLI dataset. By rank-adaptive training, CoMERA achieves
 - **80×** compression ratio
 - **2-3×** speedup
- Comparison with existing methods, GaLore and LTE.
 - **2×** faster than GaLore and **3×** faster than LTE for all batch sizes
 - **9×** more memory-efficient than GaLore in single-batch training.



Training time per epoch for the six-encoder transformer model on the MNLI dataset.



(a). Training time per epoch

(b). Peak memory consumption

Training time and total memory cost of CoMERA, GaLore and LTE on a six-encoder transformer with varying batch sizes. The experiment is done on Nvidia RTX 3090 GPU.

DLRM Model with 4-GB Model Size

- We test CoMERA to train DLRM on Criteo Ad Kaggle dataset and achieve
 - **99×** compression ratio
 - **7×** peak memory saving

	uncompressed	CoMERA
accuracy	78.68%	78.76%
normalized CE	0.793	0.792
model size (GB)	4.081	0.041 (99×)
peak memory (GB)	18.275	2.612 (7×)

Training results on the DLRM model with a batch size 10,000.